

UNIVERSITY BILINGUAL INTERNATIONAL AND SUSTAINABLE

PRACTICE 3

"IMPLEMENTATION OF ADVANCED DATA CLEANING IN PYTHON"

EXTRACTION KNOWLEDGE

PRESENTS:

ANA PAOLA IRIGOYEN JUÁREZ

23-02-004

TEACHER

JESUS LOBATO BAEZ

SAN JOSE CHIAPA, PUEBLA, PUE. SEPTEMBER, 17th, 2026.

1. Introducción y objetivo

Esta práctica implementa, en Python y desde Visual Studio Code, un flujo de limpieza de datos que (1) identifica valores inconsistentes o ruidosos, (2) detecta valores atípicos mediante el **Rango Intercuartílico (IQR)** y (3) imputa los valores faltantes con el algoritmo multivariado **KNNImputer** de scikit-learn. Como extensión, se visualizan las distribuciones antes y después del tratamiento y se compara KNN (variando *k*) contra la imputación tradicional por media y mediana.

El conjunto de datos es sintético y contiene 10 registros de clientes con tres variables: `edad`, `ingreso_mensual` y `scoring_credito`. Se introdujeron a propósito valores faltantes y dos valores erróneos (una edad de 120 años y un ingreso de $950,000).

2. Análisis inicial de calidad

La Tabla 1 muestra el dataset original. Las celdas en amarillo son valores faltantes (NaN) y las celdas en rojo son valores anómalos.

Tabla 1. Dataset original (con ruido y valores faltantes).

Registro	edad	ingreso_mensual	scoring_credito
0	25	15,000	650
1	30	22,000	700
2	NaN	18,000	680
3	45	NaN	710
4	22	12,000	590
5	120	950,000	800
6	35	25,000	670
7	28	21,000	640
8	NaN	19,000	620
9	40	NaN	730

En total hay 30 celdas: 4 son faltantes (13.3 %) y 2 son anómalas (6.7 %). Por variable:

Tabla 2. Resumen de problemas de calidad detectados en el dataset original.

Variable	Faltantes	Anómalos	Detalle
edad	2	1	Registros 2 y 8 sin dato; registro 5 con 120 años (regla de negocio: edad > 100).
ingreso_mensual	2	1	Registros 3 y 9 sin dato; registro 5 con $950,000 (límite IQR: $31,000).
scoring_credito	0	0	Completa; valores entre 590 y 800.

Un punto importante es que el **registro 5 tiene dos valores anómalos a la vez**. Al sustituirlos por NaN, ese registro conserva únicamente su `scoring_credito` (800). Por tanto, sus dos valores imputados dependerán exclusivamente de esa variable.



Figura 1. Mapa de celdas válidas, faltantes y anómalas del dataset original (izquierda) y conteo por variable (derecha).

3. Desarrollo paso a paso

3.1 Configuración del entorno

Se trabajó en un entorno virtual de Python dentro de Visual Studio Code. Las dependencias (`pandas`, `numpy`, `scikit-learn`, `scipy`, `matplotlib`, `seaborn`) se instalan con `pip install -r requirements.txt`.

3.2 Cálculo de cuartiles e IQR sobre *ingreso_mensual*

El método `quantile()` de pandas ignora los NaN, por lo que el cálculo usa los 8 ingresos observados. Ordenados de menor a mayor son: **12,000 - 15,000 - 18,000 - 19,000 - 21,000 - 22,000 - 25,000 - 950,000**. Pandas interpola linealmente en la posición $q \cdot (n - 1)$, con índices desde 0 (aquí $n = 8$, es decir, 7 intervalos):

- **Q1** ($q = 0.25$): posición $0.25 \cdot 7 = 1.75$, entre 15,000 y 18,000. Q1 = 15,000 + 0.75·(18,000 - 15,000) = **17,250**.
- **Q3** ($q = 0.75$): posición $0.75 \cdot 7 = 5.25$, entre 22,000 y 25,000. Q3 = 22,000 + 0.25·(25,000 - 22,000) = **22,750**.
- **IQR** = Q3 - Q1 = 22,750 - 17,250 = **5,500**.
- **Límite superior** = Q3 + 1.5·IQR = 22,750 + 8,250 = **31,000**.
- **Límite inferior** = Q1 - 1.5·IQR = 17,250 - 8,250 = **9,000**.

Todo ingreso mayor a $31,000 se considera atípico. El valor 950,000 lo supera ampliamente y es marcado; el resto de los ingresos (12,000 a 25,000) queda dentro de los límites, y ninguno cae por debajo de 9,000, por lo que no hay atípicos inferiores (el script aplica solo el límite superior, lo cual es suficiente en este caso). Para ilustrar el daño de un solo outlier: la media de los 8 ingresos observados es $135,250, mientras que sin el outlier es $18,857.14.

Como verificación cruzada, el mismo criterio IQR aplicado a *edad* (Q1 = 27.25, Q3 = 41.25, IQR = 14, límite superior = 62.25) también marca el valor 120. El script usa además una regla de negocio explícita (*edad* > *100*), que no depende de la distribución de los datos.

3.3 Justificación del reemplazo por NaN

Se evaluaron las alternativas para tratar los valores atípicos:

- **Eliminar el registro:** con solo 10 filas se perdería el 10 % de los datos y, además, el `scoring_credito` del registro 5, que sí es válido.
- **Recortar al límite (winsorizing):** asignaría $31,000 (o 100 años), un valor inventado en la frontera que no refleja a ningún cliente real.
- **Conservarlos:** distorsionan media, desviación estándar y la escala del gráfico, y contaminarían las distancias que usa KNN.
- **Reemplazar por NaN:** trata el dato erróneo como información no disponible y delega su estimación al imputador, que usa el perfil del cliente (otras variables) para estimarlo. Es la opción elegida.

3.4 Funcionamiento técnico de KNNImputer

`KNNImputer` completa cada valor faltante con el promedio de ese mismo atributo en los *k* registros más parecidos que sí lo tienen (los "donantes"):

- **Distancia.** Usa la distancia euclidiana que tolera NaN (`nan_euclidean`): d(x, y) = sqrt((*n*/*m*) · suma de $(x_i - y_i)^2$), donde la suma recorre solo las coordenadas presentes en ambos registros, *m* es cuántas son y *n* = 3 es el total de variables. El factor *n*/*m* compensa las coordenadas ausentes.
- **Vecinos.** Para cada celda faltante se consideran solo los registros que tienen ese atributo observado y se eligen los *k* de menor distancia.
- **Imputación.** El valor imputado es el promedio de los *k* donantes (pesos uniformes, que es el valor por defecto). Cada celda se imputa de forma independiente a partir de los datos observados.

Ejemplo con el registro 2 (edad faltante; ingreso = 18,000; scoring = 680). La Tabla 3 muestra los cinco donantes más cercanos. Con *k* = 2 los vecinos son los registros 3 y 9, y la edad imputada es (45 + 40) / 2 = **42.5**.

Tabla 3. Distancias del registro 2 a sus cinco donantes más cercanos (datos sin escalar).

Donante	edad (valor a copiar)	Coordenadas compartidas	Distancia
Registro 3	45	solo scoring	52.0
Registro 9	40	solo scoring	86.6
Registro 0	25	ingreso y scoring	3,674.4
Registro 7	28	ingreso y scoring	3,674.6
Registro 1	30	ingreso y scoring	4,899.0

Los registros 3 y 9 tienen ingreso faltante, así que su distancia al registro 2 se calcula solo con el scoring (una sola coordenada), lo que produce distancias muy pequeñas. Esto se discute en la sección 5.3.

4. Análisis de resultados

4.1 Ejecución del script

Las Figuras 2 a 4 son capturas de la ejecución de `limpieza.py` en la terminal de Visual Studio Code. Se ven el dataset original, los estadísticos del IQR (Q1 = $17,250, Q3 = $22,750, IQR = $5,500, límite superior = $31,000), el dataset tratado con NaN y los resultados de la experimentación.

```
PS C:\Users\anair\OneDrive\Documentos\Practice3> python limpieza.py
=== DATASET ORIGINAL (CON RUIDO/NULOS) ===
    edad  ingreso_mensual  scoring_credito
0   25.0         15000.0              650
1   30.0         22000.0              700
2    NaN         18000.0              680
3   45.0             NaN              710
4   22.0         12000.0              590
5  120.0        950000.0              800
6   35.0         25000.0              670
7   28.0         21000.0              640
8    NaN         19000.0              620
9   40.0             NaN              730
--------------------------------------------------
Q1: $17,250.00
Q3: $22,750.00
IQR: $5,500.00
Límite Superior para Outliers en Ingreso: $31,000.00
```

Figura 2. Captura: dataset original y cálculo de Q1, Q3, IQR y límite superior.

```
=== DATASET TRATADO (OUTLIERS E INCONSISTENCIAS REEMPLAZADOS POR NaN) ===
   edad  ingreso_mensual  scoring_credito
0  25.0         15000.0              650
1  30.0         22000.0              700
2   NaN         18000.0              680
3  45.0             NaN              710
4  22.0         12000.0              590
5   NaN             NaN              800
6  35.0         25000.0              670
7  28.0         21000.0              640
8   NaN         19000.0              620
9  40.0             NaN              730
--------------------------------------------------
[+] Gráfico guardado exitosamente como 'comparativa_outliers.png'

================================================
=== EXPERIMENTACIÓN: KNNImputer (k variable) vs MEDIA vs MEDIANA ===
================================================

--- COMPARATIVA DE VALORES IMPUTADOS: EDAD (Registros con NaN) ---
   Scoring  Media  Mediana  KNN (k=1)  KNN (k=2)  KNN (k=3)  KNN (k=5)
2      680  32.14     30.0       45.0       42.5      36.67       33.6
5      800  32.14     30.0       40.0       42.5      38.33       35.0
8      620  32.14     30.0       45.0       42.5      37.67       33.6
```

Figura 3. Captura: dataset tratado (outliers reemplazados por NaN) y comparativa de imputación para *edad*.

```
--- COMPARATIVA DE VALORES IMPUTADOS: INGRESO MENSUAL (Registros con NaN) ---
   Scoring     Media  Mediana  KNN (k=1)  KNN (k=2)  KNN (k=3)  KNN (k=5)
3      710  18857.14  19000.0    22000.0    23500.0   21666.67    20200.0
5      800  18857.14  19000.0    22000.0    20000.0   21666.67    20200.0
9      730  18857.14  19000.0    22000.0    23500.0   21666.67    20200.0

=== DATASET FINAL LIMPIO E IMPUTADO (KNN k=2) ===
   edad  ingreso_mensual  scoring_credito
0  25.0          15000.0            650.0
1  30.0          22000.0            700.0
2  42.5          18000.0            680.0
3  45.0          23500.0            710.0
4  22.0          12000.0            590.0
5  42.5          20000.0            800.0
6  35.0          25000.0            670.0
7  28.0          21000.0            640.0
8  42.5          19000.0            620.0
9  40.0          23500.0            730.0
```

Figura 4. Captura: comparativa para *ingreso mensual* y dataset final imputado (KNN, *k* = 2).

4.2 Visualización antes y después del tratamiento

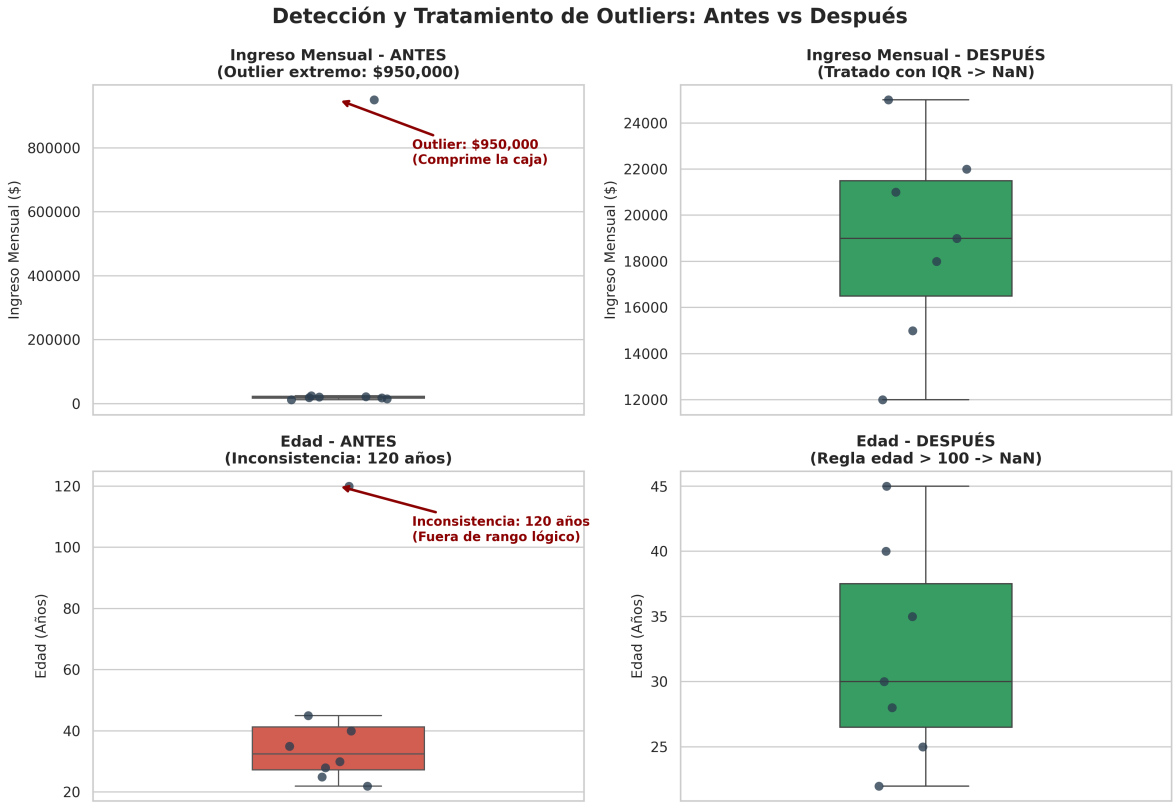


Figura 5. Boxplots de *ingreso_mensual* y *edad* antes (rojo) y después (verde) del tratamiento.

Interpretación. En el ingreso, el valor de $950,000 estira el eje hasta casi un millón y comprime la caja contra el fondo, de modo que la distribución real (12,000 a 25,000) es ilegible. Tras el tratamiento, el eje se ajusta al rango real, la mediana es $19,000 y se distingue la dispersión de los datos. En la edad, el 120 es un punto aislado muy por encima de los demás; al retirarlo la mediana pasa de 32.5 a 30 años y el rango queda entre 22 y 45 años.

4.3 Dataset ruidoso original vs. dataset final imputado

Tabla 4. Original vs. final (KNN, *k* = 2). Las celdas verdes son valores modificados por la limpieza.

Reg.	edad (orig.)	edad (final)	ingreso (orig.)	ingreso (final)	scoring
0	25	25	15,000	15,000	650
1	30	30	22,000	22,000	700
2	NaN	42.5	18,000	18,000	680
3	45	45	NaN	23,500	710
4	22	22	12,000	12,000	590
5	120	42.5	950,000	20,000	800
6	35	35	25,000	25,000	670
7	28	28	21,000	21,000	640
8	NaN	42.5	19,000	19,000	620
9	40	40	NaN	23,500	730

Tabla 5. Estadísticos descriptivos antes y después de la limpieza (la columna "original" usa solo los valores observados, incluidos los atípicos).

Estadístico	edad original (sin NaN)	edad final	ingreso original (sin NaN)	ingreso final
Media	43.12	35.25	135,250	19,900
Mediana	32.5	37.5	20,000	20,500
Desv. estándar	31.99	8.41	329,233.63	4,060.65
Mínimo	22	22	12,000	12,000
Máximo	120	45	950,000	25,000

Interpretación. La limpieza corrige la distorsión causada por los atípicos: la media del ingreso baja de $135,250 a $19,900 y la de la edad de 43.1 a 35.3 años. También cambia la forma del dato: los registros 2, 5 y 8 reciben la misma edad (42.5), de modo que la variable queda con tres valores repetidos, algo a vigilar (ver Sección 5.3). Los ingresos imputados (23,500; 20,000; 23,500) sí difieren entre registros.

5. Experimentación: variación de *k* y comparación con media y mediana

Se imputaron los mismos valores faltantes con media, mediana y KNNImputer con *k* = 1, 2, 3 y 5. Las Tablas 6 y 7 muestran los valores que recibe cada registro con dato faltante.

Tabla 6. Valores imputados para *edad* (años).

Reg.	Scoring	Media	Mediana	KNN k=1	KNN k=2	KNN k=3	KNN k=5
2	680	32.14	30	45	42.5	36.67	33.6
5	800	32.14	30	40	42.5	38.33	35
8	620	32.14	30	45	42.5	37.67	33.6

Tabla 7. Valores imputados para *ingreso_mensual* ($).

Reg.	Scoring	Media	Mediana	KNN k=1	KNN k=2	KNN k=3	KNN k=5
3	710	18,857.14	19,000	22,000	23,500	21,666.67	20,200
5	800	18,857.14	19,000	22,000	20,000	21,666.67	20,200
9	730	18,857.14	19,000	22,000	23,500	21,666.67	20,200

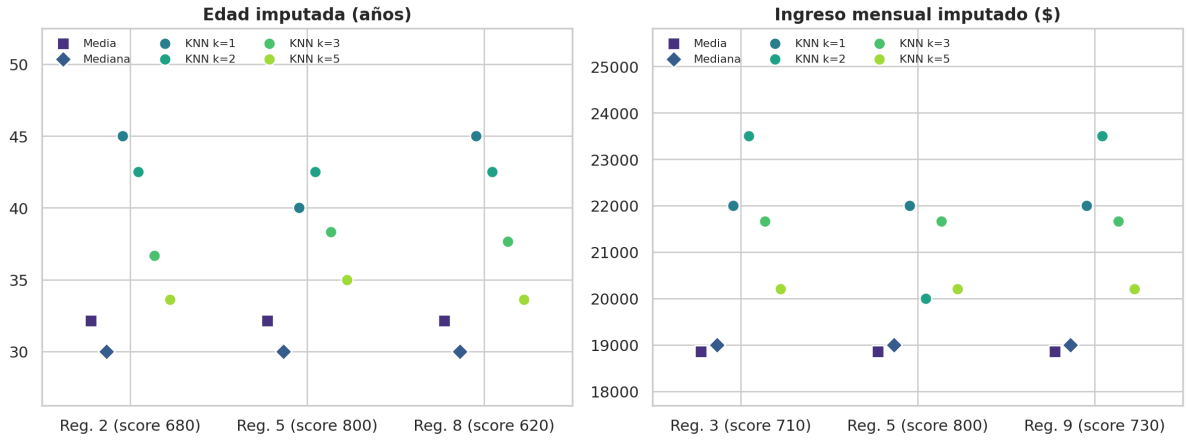


Figura 6. Valores imputados por cada método para los registros con edad (izquierda) e ingreso (derecha) faltantes.

5.1 Media y mediana

Ambos métodos asignan **el mismo valor a todos los registros** (edad 32.14 o 30; ingreso $18,857.14 o $19,000) sin importar el perfil del cliente. Por ejemplo, un cliente con scoring de 800 recibe el mismo ingreso que uno con scoring de 620. La mediana es algo más robusta, pero sigue siendo un único número.

5.2 Efecto del número de vecinos *k*

- **k = 1:** copia el valor del vecino más cercano. En el ingreso, los tres registros reciben $22,000 (todos copian al registro 1) y en la edad se copian 45, 40 y 45 años. Es el valor más específico, pero depende de un único dato y por eso es el más sensible al ruido.
- **k = 2 y k = 3:** promedian 2 o 3 donantes. Los valores se suavizan y empiezan a diferenciarse entre registros (por ejemplo, el ingreso del registro 5 es 20,000 con *k* = 2 y 21,666.67 con *k* = 3).
- **k = 5:** los valores se acercan a la media y mediana (ingreso 20,200 y edad 33.6 a 35), y los tres ingresos imputados llegan a ser idénticos. Cuando *k* iguala el número de donantes disponibles (7 en este dataset), KNN se reduce exactamente a la media; esto se comprobó numéricamente en el código.

En resumen, *k* pequeño da más varianza (valores locales pero inestables) y *k* grande da más sesgo hacia la media global (valores estables pero poco personalizados).

5.3 Observación técnica: escalas y distancias parciales

Con *k* = 2 los registros 2, 5 y 8 reciben exactamente la misma edad (42.5). Al revisar los vecinos se identificaron dos causas que explican este comportamiento:

- **Variables con escalas muy distintas.** Cuando ambos registros tienen ingreso, las diferencias en pesos (miles) dominan la distancia frente a las de edad (decenas) y scoring (cientos). Sin escalar, el algoritmo en la práctica compara por ingreso.

- **Distancias con pocas coordenadas.** Los registros 3 y 9 (ingreso faltante) se comparan con el registro 2 solo mediante el scoring, lo que da distancias de 52 y 87, frente a más de 3,600 para los registros con ingreso observado (Tabla 3). Así, para los registros 2 y 8 quedan como vecinos más cercanos los registros 3 y 9, aunque haya registros con ingreso casi idéntico (por ejemplo, el registro 7 con $21,000 y 28 años para el registro 8 de $19,000).

Como prueba adicional (incluida en `extension_analisis.py`), se repitió la imputación escalando las variables al rango [0, 1] con `MinMaxScaler` y devolviéndolas a su escala original. Los resultados se muestran en la Tabla 8.

Tabla 8. KNNImputer con variables escaladas (MinMax) frente al resultado sin escalar para *k* = 2.

Variable / registro	k=1	k=2	k=3	k=5	k=2 sin escalar
edad / reg. 2	45	35	32.67	33.6	42.5
edad / reg. 5	40	42.5	38.33	35	42.5
edad / reg. 8	28	26.5	27.67	28	42.5
ingreso / reg. 3	18,000	21,500	20,666.67	21,000	23,500
ingreso / reg. 5	22,000	20,000	21,666.67	20,200	20,000
ingreso / reg. 9	18,000	21,500	21,666.67	21,000	23,500

Con el escalado, las edades imputadas con *k* = 2 pasan a 35, 42.5 y 26.5 (antes 42.5 en los tres casos), es decir, el resultado sí depende del preprocesamiento. El registro 5 no cambia porque solo dispone del scoring. El escalado no elimina del todo el efecto de las distancias con pocas coordenadas: por ejemplo, con *k* = 1 el ingreso de los registros 3 y 9 es 18,000. Sin valores reales con los cuales comparar, **no puede afirmarse cuál de las dos versiones es más exacta**; para ello habría que ocultar valores conocidos y medir el error de reconstrucción, algo poco significativo con solo 10 registros. Lo que sí queda demostrado es que, con KNN, escalar las variables es una decisión que debe tomarse y justificarse.

6. Conclusiones

Ventajas de la imputación con KNN frente a media o mediana

- Usa la información de las demás variables: la imputación depende del perfil del registro, no es un único valor para todos (en el ingreso, KNN dio 20,000 y 23,500 donde la mediana daba 19,000 para todos).
- Conserva mejor la variabilidad y la relación entre variables, mientras que la media y la mediana la reducen y debilitan la covarianza.
- Es no paramétrico: no supone una distribución determinada y la elección de *k* permite ajustar el equilibrio entre especificidad y estabilidad.

Desventajas y precauciones

- Es sensible a las escalas: sin normalizar, la variable de mayor magnitud domina la distancia (Sección 5.3).
- Con datos faltantes, las distancias se calculan con pocas coordenadas, lo cual puede favorecer a donantes poco informativos.
- Es sensible a los outliers: por eso se tratan antes con IQR y regla de negocio. Un dato atípico usado como donante contaminaría la imputación.

- Requiere elegir *k* y es más costoso computacionalmente (calcula distancias entre registros), mientras que media y mediana son inmediatas y reproducibles.
- Con pocos datos (aquí 10 registros) los vecinos son pocos y los resultados cambian mucho con *k*; los valores imputados son estimaciones, no datos reales.

Cierre. El flujo implementado (regla de negocio y IQR para detectar atípicos, reemplazo por NaN e imputación KNN) produjo un dataset completo y sin valores absurdos. La media del ingreso pasó de $135,250 a $19,900. KNN aportó imputaciones diferenciadas por perfil, pero su resultado depende de *k* y del escalado, por lo que conviene validarlo (por ejemplo, con validación cruzada sobre datos ocultos) antes de usarlo en un caso real.

Anexo. Código fuente (limpieza.py)

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.impute import KNNImputer, SimpleImputer

# ========================================
# 1. Generación de dataset con ruido sintético
# ========================================
data = {
    'edad': [
        25,
        30,
        np.nan,
        45,
        22,
        120,
        35,
        28,
        np.nan,
        40,
    ],  # 120 es valor atípico/erróneo
    'ingreso_mensual': [
        15000,
        22000,
        18000,
        np.nan,
        12000,
        950000,
        25000,
        21000,
        19000,
        np.nan,
    ],  # 950k es outlier
    'scoring_credito': [650, 700, 680, 710, 590, 800, 670, 640, 620, 730],
}

# Guardamos copia del dataset original antes de modificarlo
df_original = pd.DataFrame(data)
df_dirty = df_original.copy()

print("=== DATASET ORIGINAL (CON RUIDO/NULOS) ===")
print(df_dirty)
print("-" * 50)

# ========================================
# 2. Tratamiento de Valores Atípicos (Outliers e Inconsistencias)
# ========================================
# Cálculo de rango intercuartílico (IQR) para ingreso_mensual
Q1 = df_dirty['ingreso_mensual'].quantile(0.25)
print(f"Q1: ${Q1:,.2f}")
Q3 = df_dirty['ingreso_mensual'].quantile(0.75)
print(f"Q3: ${Q3:,.2f}")
IQR = Q3 - Q1
print(f"IQR: ${IQR:,.2f}")
limite_superior = Q3 + 1.5 * IQR

print(f"Límite Superior para Outliers en Ingreso: ${limite_superior:,.2f}")

# Reemplazar outliers extremos de ingresos con NaN para tratarlos en la imputación
df_dirty.loc[df_dirty['ingreso_mensual'] > limite_superior, 'ingreso_mensual'] = (
    np.nan
)

# Corrección de regla de negocio: Edad mayor a 100 se convierte a NaN
df_dirty.loc[df_dirty['edad'] > 100, 'edad'] = np.nan

print("\n=== DATASET TRATADO (OUTLIERS E INCONSISTENCIAS REEMPLAZADOS POR NaN) ===")
print(df_dirty)
print("-" * 50)

# ========================================
# 3. Visualización Gráfica: Antes y Después del Tratamiento de Outliers
# ========================================
```

```
sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(2, 2, figsize=(13, 9))
fig.suptitle(
    'Detección y Tratamiento de Outliers: Antes vs Después',
    fontsize=16,
    fontweight='bold',
    y=0.98,
)

color_antes = '#e74c3c'    # Rojo suave
color_despues = '#27ae60'  # Verde suave

# --- Fila 1: Ingreso Mensual ---
# 1.1 Ingreso Mensual: Antes
sns.boxplot(ax=axes[0, 0], y=df_original['ingreso_mensual'], color=color_antes, width=0.35, showfliers=False)
sns.stripplot(ax=axes[0, 0], y=df_original['ingreso_mensual'], color='#2c3e50', alpha=0.8, jitter=0.1, size=7)
axes[0, 0].set_title('Ingreso Mensual - ANTES\n(Outlier extremo: $950,000)', fontsize=12, fontweight='bold')
axes[0, 0].set_ylabel('Ingreso Mensual ($)', fontsize=11)
axes[0, 0].annotate(
    'Outlier: $950,000\n(Comprime la caja)',
    xy=(0, 950000),
    xytext=(0.15, 750000),
    arrowprops=dict(facecolor='darkred', edgecolor='darkred', arrowstyle='->', lw=2),
    fontsize=10,
    fontweight='bold',
    color='darkred'
)

# 1.2 Ingreso Mensual: Después
sns.boxplot(ax=axes[0, 1], y=df_dirty['ingreso_mensual'], color=color_despues, width=0.35, showfliers=False)
sns.stripplot(ax=axes[0, 1], y=df_dirty['ingreso_mensual'], color='#2c3e50', alpha=0.8, jitter=0.1, size=7)
axes[0, 1].set_title('Ingreso Mensual - DESPUÉS\n(Tratado con IQR -> NaN)', fontsize=12, fontweight='bold')
axes[0, 1].set_ylabel('Ingreso Mensual ($)', fontsize=11)

# --- Fila 2: Edad ---
# 2.1 Edad: Antes
sns.boxplot(ax=axes[1, 0], y=df_original['edad'], color=color_antes, width=0.35, showfliers=False)
sns.stripplot(ax=axes[1, 0], y=df_original['edad'], color='#2c3e50', alpha=0.8, jitter=0.1, size=7)
axes[1, 0].set_title('Edad - ANTES\n(Inconsistencia: 120 años)', fontsize=12, fontweight='bold')
axes[1, 0].set_ylabel('Edad (Años)', fontsize=11)
axes[1, 0].annotate(
    'Inconsistencia: 120 años\n(Fuera de rango lógico)',
    xy=(0, 120),
    xytext=(0.15, 102),
    arrowprops=dict(facecolor='darkred', edgecolor='darkred', arrowstyle='->', lw=2),
    fontsize=10,
    fontweight='bold',
    color='darkred'
)

# 2.2 Edad: Después
sns.boxplot(ax=axes[1, 1], y=df_dirty['edad'], color=color_despues, width=0.35, showfliers=False)
sns.stripplot(ax=axes[1, 1], y=df_dirty['edad'], color='#2c3e50', alpha=0.8, jitter=0.1, size=7)
axes[1, 1].set_title('Edad - DESPUÉS\n(Regla edad > 100 -> NaN)', fontsize=12, fontweight='bold')
axes[1, 1].set_ylabel('Edad (Años)', fontsize=11)

plt.tight_layout()
plt.savefig('comparativa_outliers.png', dpi=300)
print("[+] Gráfico guardado exitosamente como 'comparativa_outliers.png'")
# Descomenta la siguiente línea si deseas abrir la ventana interactiva al ejecutar:
# plt.show()

# ========================================
# 4. Experimentación: Variación de k en KNNImputer vs Métodos Tradicionales
# ========================================
print("\n" + "=" * 50)
print("=== EXPERIMENTACIÓN: KNNImputer (k variable) vs MEDIA vs MEDIANA ===")
print("=" * 50)

# Imputación por Media y Mediana (métodos tradicionales univariados)
imp_media = SimpleImputer(strategy='mean').fit_transform(df_dirty)
df_media = pd.DataFrame(imp_media, columns=df_dirty.columns)

imp_mediana = SimpleImputer(strategy='median').fit_transform(df_dirty)
```

```
df_mediana = pd.DataFrame(imp_mediana, columns=df_dirty.columns)

# Imputación con KNN para distintos valores de k (k=1, k=2, k=3, k=5)
valores_k = [1, 2, 3, 5]
dict_knn = {}

for k in valores_k:
    imputer_k = KNNImputer(n_neighbors=k)
    dict_knn[f'KNN (k={k})'] = pd.DataFrame(imputer_k.fit_transform(df_dirty), columns=df_dirty.columns)

# Índices donde había valores faltantes en 'edad' e 'ingreso_mensual'
idx_edad_nulos = df_dirty[df_dirty['edad'].isna()].index
idx_ingreso_nulos = df_dirty[df_dirty['ingreso_mensual'].isna()].index

# Tabla comparativa para la columna EDAD
comp_edad = pd.DataFrame({
    'Scoring': df_dirty.loc[idx_edad_nulos, 'scoring_credito'],
    'Media': df_media.loc[idx_edad_nulos, 'edad'],
    'Mediana': df_mediana.loc[idx_edad_nulos, 'edad'],
    **{nombre: dict_knn[nombre].loc[idx_edad_nulos, 'edad'] for nombre in dict_knn}
})

print("\n--- COMPARATIVA DE VALORES IMPUTADOS: EDAD (Registros con NaN) ---")
print(comp_edad.round(2))

# Tabla comparativa para la columna INGRESO MENSUAL
comp_ingreso = pd.DataFrame({
    'Scoring': df_dirty.loc[idx_ingreso_nulos, 'scoring_credito'],
    'Media': df_media.loc[idx_ingreso_nulos, 'ingreso_mensual'],
    'Mediana': df_mediana.loc[idx_ingreso_nulos, 'ingreso_mensual'],
    **{nombre: dict_knn[nombre].loc[idx_ingreso_nulos, 'ingreso_mensual'] for nombre in dict_knn}
})

print("\n--- COMPARATIVA DE VALORES IMPUTADOS: INGRESO MENSUAL (Registros con NaN) ---")
print(comp_ingreso.round(2))

# Dataset final limpio con el modelo base (k=2)
df_clean = dict_knn['KNN (k=2)']
print("\n=== DATASET FINAL LIMPIO E IMPUTADO (KNN k=2) ===")
print(df_clean)

# ========================================
# 5. Breve Comparativa y Conclusiones Teóricas
# ========================================
print("\n" + "=" * 50)
print("=== RESUMEN COMPARATIVO DE MÉTODOS DE IMPUTACIÓN ===")
print("=" * 50)
print("""
1. Métodos Tradicionales (Media / Mediana):
   - Asignan un valor idéntico y estático a todas las filas faltantes (ej. Media ingreso: $18,857.14, Mediana:
     $19,000.00).
   - Ignoran la información de las otras variables del individuo (por ejemplo, el scoring crediticio).
   - Reducen artificialmente la variabilidad y distorsionan la covarianza entre características.

2. KNNImputer (Multivariado):
   - Imputa valores basándose en individuos con perfiles similares (distancia euclidiana sobre variables disponibles
     como scoring_credito).
   - Para un individuo con scoring alto (ej. 800), busca donantes cercanos en el espacio multidimensional en lugar de
     asignar el promedio global.

3. Efecto de modificar k en KNN:
   - k=1: Asigna exactamente el valor del vecino más próximo. Puede capturar particularidades locales, pero es muy
     sensible a anomalías o ruido puntual.
   - k=2 y k=3: Promedia los 2 o 3 vecinos más parecidos, logrando un balance entre especificidad local y estabilidad
     numérica.
   - k grande (k=5): Al incluir más vecinos dispersos, las imputaciones se suavizan y tienden a converger hacia la
     media del conjunto.
""")
```